

TRABAJO PRÁCTICO INTEGRADOR 2026

Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

Informe Técnico Analítico

Paradigmas y Lenguajes de Programación

Grupo: 39

Lenguaje asignado: Erlang

Fecha de entrega: 16 de junio de 2026

Integrantes:

- | | |
|----------------------------------|--|
| • Pereira Ignacio Valentín | - Usuario GitHub: ignacvpr |
| • Juliana Turbay | - Usuario GitHub: juliturbay |
| • Miranda Francisco Martín | - Usuario GitHub: franDeveloper1 |
| • Pérez Rollheiser Cruz Benjamín | - Usuario GitHub: cruzperez05 |
| • Pujol Facundo Emiliano | - Usuario GitHub: facundoemilianopujol02-maker |

Introducción

Las ciudades modernas requieren sistemas de tráfico inteligentes para optimizar el flujo vehicular y garantizar la seguridad vial. Este informe presenta el desarrollo del Sistema de Semáforos Inteligentes, implementado en Common Lisp aplicando el paradigma funcional. El sistema modela las transiciones de estado del semáforo, calcula automáticamente el color activo según el tiempo, y genera un registro de los cambios para su análisis posterior.

A lo largo del informe se explica el diseño de cada función (diferenciando entre funciones puras e impuras), la integración de la librería local-time, los errores encontrados durante el desarrollo y cómo se resolvieron, y finalmente un análisis comparativo con el lenguaje Erlang, asignado para la Fase 3.

Fase 1

Desarrollo de la Fase 1: El Problema de los 3 Focos y Control Urbano

Enfoque de Diseño Metodológico bajo Restricciones Funcionales

El desarrollo del Sistema de Semáforos Inteligentes respetó un conjunto de restricciones de diseño impuestas por la cátedra, con el objetivo de garantizar un enfoque puramente funcional:

Inmutabilidad Absoluta: No se utilizaron variables globales mutables (`defparameter`, `defvar`) ni operadores destructivos (`setq`, `setf`). El estado del sistema (el tiempo Unix y el color actual del semáforo) no se almacena en ninguna variable, sino que se pasa como argumento de una función a otra cada vez que se necesita.

Ausencia de Bucles Imperativos: Quedó estrictamente prohibida la utilización de estructuras iterativas tradicionales como `loop`, `dolist`, `dotimes` o `while`. La simulación del paso del tiempo y el procesamiento secuencial de colecciones de datos se resolvieron de forma puramente declarativa mediante evaluación condicional basada en funciones primitivas de Common Lisp y funciones de orden superior (`mapcar`), respectivamente.

Justificación Arquitectónica de la Extensión de Seguridad

Como parte de la Iteración 2, el grupo incorporó en el archivo *amarillo-intermitente.lisp* una extensión de seguridad que modifica el comportamiento del *timer*, agregando 3 segundos de intermitencia entre cada cambio de luz.

Esta extensión agrega el nuevo estado *'en-amarillo-intermitente* y redefine el ciclo a un período total de 222 segundos (216 + 3 + 3 segundos de intermitencia), mediante aritmética modular (*mod tiempo-unix 222*). Esto demuestra que el diseño funcional permite agregar nueva complejidad sin modificar ni romper la estructura base del sistema.

Búsqueda de Bugs

Bug 1 — Error con el formato de FORMAT

El problema es que ~S muestra los símbolos en mayúsculas y con comillas internas, mientras que ~A los muestra de forma más limpia. Pero más grave: si el formato esperado es exactamente "Tiempo <epoch>: la luz ha cambiado de <color> a <color>", usar ~S puede mostrar los símbolos con comillas simples en algunos compiladores:

Tiempo 1715868000: la luz ha cambiado de 'EN-ROJO a 'EN-VERDE

Solución: Cambiando ~S por ~A: (defun log-cambio-estado (epoch color-anterior color-nuevo) (format t "Tiempo A: la luz ha cambiado de ~A a ~A%" epoch color-anterior color-nuevo))

```
1]> (defun ciclos-por-tiempo (duracion-minutos)
  (let* ((segundos-totales (* duracion-minutos 60))
        (duracion-ciclo (+ 90 6 120))
        (ciclos-completos (round (/ segundos-totales duracion-ciclo))))
    ciclos-completos))

CICLOS-POR-TIEMPO
2]> (ciclos-por-tiempo 20)

5

3]> (defun ciclos-por-tiempo (duracion-minutos)
  (let* ((segundos-totales (* duracion-minutos 60))
        (duracion-ciclo (+ 90 6 120))
        (ciclos-completos (floor (/ segundos-totales duracion-ciclo))))
    ciclos-completos))

CICLOS-POR-TIEMPO
4]> (ciclos-por-tiempo 20)
```

Bug 2 — Error al validar con MEMBER y símbolos sin apóstrofe Al escribir la función `color-valido-p`, olvidamos el apóstrofe en la lista de colores válidos.

CÓDIGO CON BUG

```
(defun color-valido-p (color)
```

(member color (en-rojo en-amarillo en-verde))) El compilador intenta ejecutar `en-rojo` como si fuera una función, porque sin el apóstrofe LISP evalúa todo.

Solución: Agregando el apóstrofe `'` para indicar que es una lista de datos literales:

CÓDIGO CORREGIDO

```
(defun color-valido-p (color)
```

(member color '(en-rojo en-amarillo en-verde))) cuando `(en-rojo en-amarillo en-verde)` sin apóstrofe, el intérprete intenta evaluar `en-rojo` como una función. Con el apóstrofe `'(en-rojo en-amarillo en-verde)` le decís a LISP que trate eso como un dato literal, es decir, una lista de símbolos.

```
5]> (defun ciclos-por-tiempo (duracion-minutos)
      (let* (; ERROR: usamos los minutos directamente sin convertir
            (duracion-ciclo (+ 90 6 120))
            (ciclos-completos (floor (/ duracion-minutos duracion-ciclo))))
        ciclos-completos))

CICLOS-POR-TIEMPO
6]> (ciclos-por-tiempo 15)

7]> (defun ciclos-por-tiempo (duracion-minutos)
      (let* ((segundos-totales (* duracion-minutos 60)) ; convertimos primer
            (duracion-ciclo (+ 90 6 120))
            (ciclos-completos (floor (/ segundos-totales duracion-ciclo))))
        ciclos-completos))

CICLOS-POR-TIEMPO
8]> (ciclos-por-tiempo 15)

9]> |
```

Bug 3 - Quicklisp no funciona en CLISP configurado en Sublime Text 3:

Cuando intenté usar la librería local-time en mi entorno habitual de código (CLISP con Sublime Text), escribí (ql:quickload :local-time) y ejecuté el archivo con Ctrl+Alt+H. El intérprete me tiró el error no existe ningún paquete con el nombre "QL". Investigando el problema me di cuenta que CLISP en Windows no tiene Quicklisp instalado por defecto. Intenté cargar el archivo quicklisp.lisp manualmente de varias formas:

Primero probé (load "C:/Users/1gn4c10/Desktop/quicklisp.lisp") tiro Error de Win32 267 - The directory name is invalid

Probé con doble barra (load "C:\\Users\\1gn4c10\\Desktop\\quicklisp.lisp") - mismo error

Probé con #p adelante - mismo error

Guardé el archivo en C:\\ directamente y probé (load "C:/quicklisp.lisp") - mismo error

Después de todos estos intentos fallidos concluí que CLISP en Windows no es compatible con Quicklisp de forma sencilla y tuve que buscar otra solución.

Solución - Instalación de SBCL y configuración de Quicklisp:

Descargué SBCL desde sbcl.org, específicamente la versión 2.6.5 para Windows x86-64. Lo instalé con el instalador .msi siguiendo los pasos por defecto. Una vez instalado lo abrí desde el cmd escribiendo sbcl. Para instalar Quicklisp dentro de SBCL seguí estos pasos:

Descargué el archivo quicklisp.lisp desde quicklisp.org

Lo guardé en el escritorio

En SBCL ejecuté (load "C:/Users/1gn4c10/Desktop/quicklisp.lisp") - esta vez sí funcionó y cargó Quicklisp

Ejecuté (quicklisp-quickstart:install) - descargó e instaló Quicklisp completamente

Ejecuté (ql:quickload "local-time") - descargó e instaló la librería local-time correctamente

```

*REPL* [lisp] - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
◀ ▶ prueba.lisp × *REPL* [lisp] × core.lisp × *REPL* [lisp]
[1]> (load "quicklisp.lisp")

*** - LOAD: A file with name quicklisp.lisp does not exist
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort main loop
Break 1 [2]> (load "C:\\Users\\1gn4c10\\Desktop\\quicklisp.lisp")

*** - Error de Win32 267 (ERROR_DIRECTORY): The directory name is invalid.
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort main loop
Break 2 [3]>

```

```

C:\Windows\system32\cmd.exe
; Fetching #<URL "http://beta.quicklisp.org/client/2021-02-13/quicklisp.tar">
; 260.00KB
=====
266,240 bytes in 0.19 seconds (1396.20KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/client/2021-02-11/setup.lisp">
; 4.94KB
=====
5,057 bytes in 0.00 seconds (2094.83KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/asdf/3.2.1/asdf.lisp">
; 628.18KB
=====
643,253 bytes in 0.44 seconds (1441.75KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/dist/quicklisp.txt">
; 0.40KB
=====
488 bytes in 0.00 seconds (168.12KB/sec)
Installing dist "quicklisp" version "2026-01-01".
; Fetching #<URL "http://beta.quicklisp.org/dist/quicklisp/2026-01-01/releases.txt">
; 564.04KB
=====
577,575 bytes in 0.63 seconds (897.65KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/dist/quicklisp/2026-01-01/systems.txt">
; 458.96KB
=====
469,975 bytes in 0.17 seconds (2701.27KB/sec)

==== quicklisp installed ====

To load a system, use: (ql:quickload "system-name")

To find systems, use: (ql:system-apropos "term")

To load Quicklisp every time you start Lisp, use: (ql:add-to-init-file)

For more information, see http://www.quicklisp.org/beta/

C:\Windows\system32\cmd.exe
Microsoft Windows [Versión 10.0.26100.2605]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\lgn4cl0>sbcl
This is SBCL 2.6.5, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (load "C:/Users/lgn4cl0/Desktop/quicklisp.lisp")

==== quicklisp quickstart 2015-01-28 loaded ====

To continue with installation, evaluate: (quicklisp-quickstart:install)

For installation options, evaluate: (quicklisp-quickstart:help)

T
* (quicklisp-quickstart:install)
; Fetching #<URL "http://beta.quicklisp.org/client/quicklisp.sexp">
; 0.82KB
=====
839 bytes in 0.00 seconds (454.68KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/client/2021-02-13/quicklisp.tar">
; 260.00KB
=====
266,240 bytes in 0.19 seconds (1396.20KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/client/2021-02-11/setup.lisp">
; 4.94KB
=====
5,057 bytes in 0.00 seconds (2094.83KB/sec)
; Fetching #<URL "http://beta.quicklisp.org/asdf/3.2.1/asdf.lisp">
; 628.18KB
=====
643,253 bytes in 0.44 seconds (1441.75KB/sec)

```


Bug 4 - Timer es palabra reservada en SBCL:

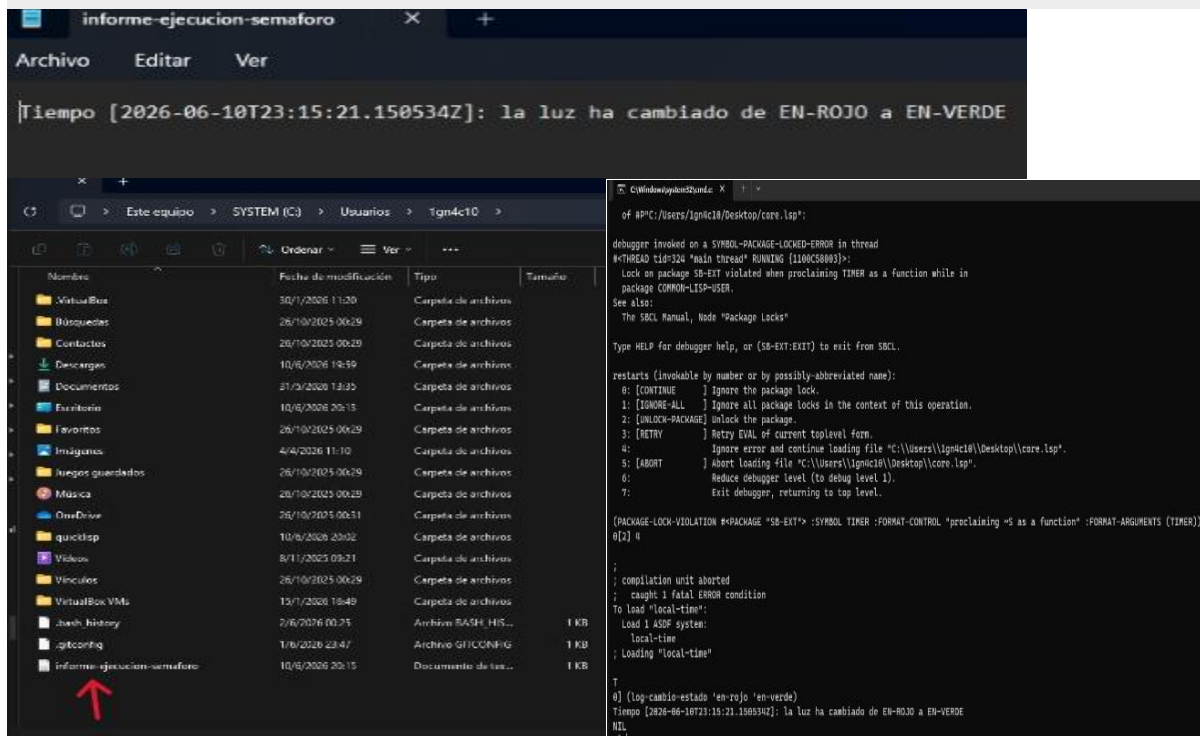
Una vez configurado todo, intenté cargar el core.lsp con (load "C:/Users/1gn4c10/Desktop/core.lsp"). SBCL tiró el siguiente error:

Lock on package SB-EXT violated when proclaiming TIMER as a function while in package COMMON-LISP-USER

El problema era que timer es un nombre reservado dentro del paquete SB-EXT de SBCL. El intérprete no permite definir una función con ese nombre sin antes desbloquear el paquete explícitamente. SBCL me ofreció varias opciones para continuar:

- 0: Ignorar el lock del paquete
- 1: Ignorar todos los locks
- 2: Desbloquear el paquete
- 3: Reintentar
- 4: Ignorar el error y continuar cargando el archivo
- 5: Abortar la carga

Seleccioné la opción 4 escribiendo 4 y enter. El archivo continuó cargando correctamente, local-time se cargó automáticamente y todas las funciones quedaron disponibles. Verifiqué ejecutando (log-cambio-estado 'en-rojo 'en-verde) que devolvió correctamente Tiempo [2026-06-10T23:15:21.150534Z]: la luz ha cambiado de EN-ROJO a EN-VERDE y además guardó el registro en el archivo informe-ejecucion-semaforo.tex.



Fase 2

¿Qué es Quicklisp?

Quicklisp es el gestor de paquetes de Common Lisp. Permite instalar librerías externas descargando un archivo de instalación, cargándolo en el intérprete y ejecutando un comando para instalar la librería deseada.

¿Por qué elegimos local-time?

Se evaluaron las dos opciones disponibles: local-time y cl-json. Se optó por local-time porque el resultado es inmediato y visible, el sistema de auditoría pasa a mostrar fechas legibles en lugar del número epoch. La opción cl-json requería además mantener un archivo de configuración externo, lo que sumaba complejidad sin aportar un beneficio significativo.

Instalación

Durante la instalación se encontraron inconvenientes con CLISP en Windows, que no era compatible con Quicklisp. Se intentaron distintas formas de cargar el archivo de instalación, pero todas resultaron en errores de directorio inválido. Como solución se instaló SBCL, otro intérprete de Common Lisp que sí es compatible con Quicklisp en Windows, y desde ahí se instaló local-time sin inconvenientes.

Modificación del sistema de auditoría

Se tomó la función log-cambio-estado ya implementada y se le realizaron dos modificaciones: se reemplazó el parámetro epoch por local-time:now que obtiene la fecha actual automáticamente en formato legible, y se agregó escritura en el archivo informe-ejecucion-semaforo.txt como parte de la Extensión 2.

Fase 3

Presentación del lenguaje

Erlang es un lenguaje de programación creado en 1986 por Ericsson, empresa sueca de telecomunicaciones. Fue desarrollado con el objetivo de construir sistemas que no pudieran interrumpirse, si una parte falla, el resto debe continuar funcionando. Su principal característica es la tolerancia a fallos y la capacidad de manejar miles de procesos de forma simultánea.

Industrias y empresas que lo usan

Se utiliza principalmente en telecomunicaciones, mensajería y sistemas distribuidos. WhatsApp lo adoptó para gestionar millones de mensajes simultáneos con una infraestructura reducida. Discord también lo utiliza para sostener millones de usuarios conectados al mismo tiempo.

Pregunta 1 -Sintaxis de Erlang comparada con Lisp

En Lisp todas las expresiones se delimitan con paréntesis. En Erlang se utilizan tres símbolos de puntuación con roles distintos: la coma “,” separa expresiones dentro de una cláusula, el punto y coma “;” separa cláusulas de la misma función, y el punto “.” indica el fin de la función. Esto le da al código una estructura más cercana a la notación matemática.

Otra diferencia relevante es el manejo de casos. En Lisp se utiliza cond dentro de una sola función. En Erlang se define la misma función varias veces, una por cada caso posible, y el lenguaje selecciona automáticamente cuál ejecutar según los parámetros recibidos. Este mecanismo se denomina pattern matching.

Pregunta 2 - Cómo simula Erlang ciclos continuos sin reasignar variables

En Erlang una variable no puede modificar su valor una vez asignado. Para simular el ciclo continuo del semáforo se utiliza recursión, la función se llama a sí misma con el nuevo estado como parámetro en lugar de modificar una variable existente. De esta forma el ciclo continúa sin necesidad de reasignar valores.

Conclusión sobre Erlang

Erlang resultó ser un lenguaje considerablemente diferente a Common Lisp. La sintaxis basada en puntos y comas fue lo más difícil de incorporar al principio, especialmente viniendo de trabajar con paréntesis. El concepto de pattern matching también requirió un tiempo de adaptación, ya que implica pensar la función de una manera distinta. Una vez comprendidos estos conceptos, el código resultó más claro y directo. Fue una experiencia útil para entender que un mismo problema puede resolverse de maneras muy distintas según el lenguaje.

Conclusión

Este trabajo nos permitió poner en práctica los conceptos del paradigma funcional que vimos en la materia: la inmutabilidad, la diferencia entre funciones puras e impuras, y resolver problemas con recursión y funciones de orden superior en vez de variables y bucles. Comparar con Erlang nos ayudó a ver que un mismo problema, como las transiciones del semáforo, se puede resolver de formas muy distintas según el lenguaje: con cond en Lisp, o con pattern matching en Erlang.

En cuanto a la experiencia, lo más difícil no fue tanto pensar de forma funcional, sino adaptarse a la sintaxis y al entorno de Common Lisp, algo que se fue resolviendo a medida que avanzaba el trabajo y se detallan en la bitácora de bugs. Más allá de esas dificultades, el resultado final cumple con lo solicitado y nos deja una base sólida para entender cómo se piensa un programa cuando no se depende de variables ni ciclos tradicionales.

Bibliografía

Quicklisp: Quicklisp beta
Librería local-time: LOCAL-TIME :: A Common Lisp Library for Times and Dates
SBCL: About - Steel Bank Common Lisp
Documentación de Erlang: https://www.erlang.org/doc/
Try Erlang (tutorial online): Try Erlang